WIZM

A Post-Quantum Payment Chain with Transparent and Unlinkable Stealth Transfers
Protocol Design, Consensus, Operations, and Evaluation

WIZM Protocol Engineering

29 October 2026

Abstract

WIZM is a deterministic, post-quantum payment ledger implemented in Rust. The current network is a permissioned proof-of-authority chain with five validator seats, ten-second steady-state slots, full block bodies, and two transaction families: transparent transfers and stealth transfers. Falcon-512 signs transactions, block seals, notarization votes, and transport envelopes; ML-KEM-512 protects committee and mempool key establishment; ML-KEM-768 protects inter-validator transport; BLAKE3 provides hashing and address derivation; AES-256-GCM protects encrypted mempool payloads. A block carries at most 1,200 transactions and 8 MiB, giving a sealed throughput ceiling of 120 transactions per second. Fees are flat at 10,000 base units for the first 20 fee-paying transactions from a wallet in each block, then rise through a deterministic per-wallet burst schedule. During the emission period fees are burned; after emission they are distributed to validators. Stealth transfers hide the sender-to-recipient link through a one-time recipient address; sender and amount remain public consensus data. This paper describes the protocol, threat model, node architecture, consensus and finality rules, mempool design, storage and synchronization path, RPC and wallet boundaries, tokenomics, operational limits, and the engineering evidence collected from the five-validator test network.

Contents

1 Introduction **3**
- 1.1 Contributions . . . 3
- 1.2 Scope . . . 4

2 System Model and Goals **4**
- 2.1 Roles . . . 4
- 2.2 Assumptions . . . 4
- 2.3 Goals and non-goals . . . 5

3 Node Architecture **5**
- 3.1 Startup and recovery . . . 5
- 3.2 Deployment topology . . . 6

4 Cryptographic Foundations **6**
- 4.1 Signatures: Falcon-512 . . . 6
- 4.2 Key establishment: ML-KEM . . . 6
- 4.3 Hashing and domain separation . . . 6
- 4.4 Key custody . . . 7

5 Accounts and State **7**
- 5.1 Accounts . . . 7
- 5.2 Value and units . . . 7
- 5.3 State root . . . 7

6 Transactions **7**
6.1 Transparent transfer . . . 7
6.2 Stealth transfer . . . 8
6.3 Transaction classification . . . 8
6.4 Admission policy . . . 8

7 Encrypted Mempool **9**
7.1 Entry format . . . 9
7.2 Admission and capacity . . . 9
7.3 Gap buffering . . . 9
7.4 Ordering and draining . . . 9

8 Consensus **9**
8.1 Committee . . . 9
8.2 Slots . . . 10
8.3 Block validity . . . 10
8.4 Seal suppression gates . . . 10
8.5 Fork convergence . . . 11
8.6 Finality . . . 11

9 Networking and Synchronization **11**
9.1 Transport . . . 11
9.2 Authenticated envelopes . . . 11
9.3 Synchronization . . . 11
9.4 Peer limits . . . 12

10 Storage **12**

11 RPC and API Surface **12**
11.1 Transports . . . 12
11.2 Authentication . . . 12
11.3 Rate limits and status behavior . . . 13
11.4 Endpoint families . . . 13

12 Wallet Architecture **13**
12.1 Native wallet . . . 13
12.2 Browser wallet and sidecar . . . 13
12.3 Signing boundary . . . 13

13 Tokenomics **13**
13.1 Supply and emission . . . 13
13.2 Fees . . . 14
13.3 Economic invariants . . . 14

14 Performance and Capacity **14**
14.1 Theoretical limits . . . 14
14.2 Test-network measurements . . . 14
14.3 Scaling the node set . . . 15
14.4 Fault behavior . . . 15

15 Security Analysis **15**
15.1 Threat model . . . 15
15.2 Authentication and domain separation . . . 15
15.3 Resource attacks . . . 16
15.4 Unlinkability boundary . . . 16
15.5 Residual risks . . . 16

16 Testing and Operations **16**

17 Related Work **17**

18 Limitations and Future Work **17**

19 Conclusion **17**

A Protocol Parameters **17**

B Transaction Layouts **18**
B.1 Transparent . . . 18
B.2 Stealth . . . 18

C RPC Methods **19**

D Glossary **19**

1 Introduction

Public payment ledgers face a long-horizon cryptographic migration problem: the signatures and key exchanges that secure balances today may not remain secure for the lifetime of the ledger. WIZM is an engineering response to that problem. It is a complete, running payment chain whose critical signature, key-establishment, hashing, and authenticated-encryption paths use post-quantum primitives. It is deliberately narrow: WIZM transfers value; it does not attempt to become a general-purpose computation platform.

The design is organized around four principles.

1. **Explicit trust.** The current network is permissioned. Five known validators operate a proof-of-authority schedule. The protocol does not claim permissionless validator entry or Byzantine fault tolerance.

2. **Deterministic execution.** Transactions have canonical wire layouts, fixed validation rules, sequential account nonces, and a deterministic fee function. A block is a total order of independent transfers.

3. **Bounded resources.** Blocks, mempool entries, RPC bodies, encrypted payloads, peer connections, and per-wallet burst fees all have explicit ceilings. Resource exhaustion is treated as a protocol and operations problem, not as an implementation detail.

4. **Useful recipient unlinkability.** Stealth transfers replace a reusable recipient address with a one-time address derived from a published meta-address. This hides the sender-to-recipient link. It does not hide the sender or the amount, and the paper does not claim otherwise.

1.1 Contributions

This document makes the current system inspectable and reproducible. Its contributions are:

- a precise account of the post-quantum cryptographic stack and its domain-separated message formats;
- canonical transparent and stealth transaction layouts, including validation, signature coverage, size envelopes, and fee-payer rules;
- a five-validator proof-of-authority protocol with deterministic producer rotation, bounded fallback attempts, same-height convergence, and contiguous notary finality;
- a gap-buffered encrypted mempool with per-sender floors, parking, eviction, and admission backpressure;
- a distributed node architecture covering libp2p transport, gossip, unicast, range synchronization, snapshots, state persistence, and restart recovery;
- an authenticated RPC and wallet boundary with per-key and per-IP limits and explicit HTTP status behavior;
- a transparent tokenomics schedule with deterministic fee burn, post-emission validator distribution, and a per-wallet burst rule;
- a test and operations methodology, including a 50-node public-testnet target topology, fault tests, and a ten-day mixed-transaction soak.

1.2 Scope

WIZM is a payment ledger, not a general smart-contract machine. It has no virtual machine, no arbitrary stateful programs, and no on-chain governance language. The two value-bearing transaction families are described in Section 6. The current deployment is a test network and is not presented as a production mainnet.

2 System Model and Goals

2.1 Roles

A WIZM deployment consists of:

- **validators**, which produce blocks, sign seals and votes, and participate in finality;
- **full nodes**, which validate, store, and serve the chain;
- **RPC gateways**, which expose authenticated read/write APIs;
- **wallets**, which construct and sign transfers;
- **operators**, who manage keys, configuration, TLS, monitoring, and recovery.

In the current five-node network each validator also serves the chain locally. A public deployment separates validator infrastructure from public RPC and observer infrastructure.

2.2 Assumptions

The protocol assumes that:

1. at least three of the five configured validator seats are operated by trusted parties and are online for finality;
2. validator secret keys are stored with restrictive permissions and are not exposed to RPC clients;

3. clocks are loosely synchronized to wall time; slot arithmetic uses an explicit grace window to absorb propagation jitter;

4. the cryptographic primitives and their implementations behave as specified;

5. clients protect their own signing keys and API credentials.

The protocol does not claim BFT safety under arbitrary validator equivocation. It is a crash-fault design: the five-seat schedule tolerates one or two unavailable or crashed validators, and a three-validator outage halts progress until quorum returns.

2.3 Goals and non-goals

The goals are: post-quantum authentication, deterministic transfer execution, bounded latency and resource use, restart-safe state, recipient unlinkability for stealth payments, and an operable API and wallet boundary.

The non-goals are: hiding the sender or amount in stealth payments, making transparent transfers unlinkable, executing arbitrary programs, offering permissionless validation, or replacing independent security review with the claims in this paper.

3 Node Architecture

A node is a Rust process with the following internal layers.

Component	Responsibility
`wizm-node`	configuration, startup, PoA loop, operator hooks, network task, graceful shutdown
`wizm-consensus`	slot schedule, committee logic, notary, mempool, wire messages, synchronization state
`wizm-executor`	transaction classification, validation, state transition, balances, nonces, fees, key-image state
`wizm-network`	libp2p TCP/QUIC transport, encrypted envelopes, gossip, unicast, request/response ranges
`wizm-storage`	RocksDB column families, blocks, headers, state, snapshots, rewind journal, metadata
`wizm-rpc`	JSON-RPC and gRPC handlers, authentication, rate limits, request metadata and status mapping
`wizm-crypto`	Falcon-512, ML-KEM, BLAKE3, AES-GCM, addresses, stealth derivation
`wizm-tokenomics`	supply, emission, block reward, fees, burst multiplier

3.1 Startup and recovery

On startup a node loads its configuration, validator key material, peer identity, committee KEM material, and RocksDB state. It then:

1. loads the newest complete snapshot if one exists, otherwise starts from the genesis state;

2. replays stored block bodies from the executor's resume point to the stored tip;

3. derives the producer committee and anchors the wall-clock slot;

4. starts the JSON-RPC and gRPC servers and the PoA loop;

5. joins the libp2p mesh and begins gossip, sync, and vote exchange.

The mempool is in-memory and is reconstructed from new submissions after a restart. Block history, account state, key-image state, tokenomics counters, and consensus metadata are persisted.

3.2 Deployment topology

The reference topology for a public testnet is:

Role	Count
Consensus validators	5
Seed/full nodes	5
Public RPC/full nodes	10
Community observer/full nodes	30
Total target	50

The per-node peer cap is 128. Validators connect to each other and to seeds; public nodes bootstrap from seeds. Observers do not vote or produce. The 50-node target leaves substantial headroom below the per-node cap and is the basis for the load-test plan in Section 14.

4 Cryptographic Foundations

4.1 Signatures: Falcon-512

Transactions, block seals, notary votes, and transport envelopes use Falcon-512. A public key is 897 bytes; a detached signature is variable-length in the range 512–752 bytes. Verification checks the length envelope, deserializes the public key, recomputes the address where required, and verifies the signature over a domain-separated message.

The address of a public key is

$$\text{addr}(pk) = \text{BLAKE3}(pk)[0..20]. \tag{1}$$

The 20-byte address binds the signing key to the account used for balance and nonce checks.

4.2 Key establishment: ML-KEM

ML-KEM-512 is used for committee and mempool key establishment. The committee holds a KEM key pair; mempool entries are encrypted under the committee public key. ML-KEM-768 is used for inter-validator transport key establishment. Authenticated encryption uses AES-256-GCM with a per-entry nonce and authentication tag.

4.3 Hashing and domain separation

BLAKE3 provides transaction hashes, block hashes, address derivation, state accumulation, stealth derivation, and protocol message digests. Structured messages use distinct domain prefixes:

- `wizm-consensus-domain-v1` binds chain ID and genesis hash to consensus signatures;
- `wizm/vote/v2` binds wire votes to domain, block hash, and round;
- `wizm/notary/v2` binds notarization votes to domain, block hash, and height;
- separate domains bind mempool encryption, transport, and account accumulation.

4.4 Key custody

Validator keys, committee KEM keys, peer identity keys, TLS keys, RPC API keys, and wallet keys are distinct. RPC keys grant API permissions only; they cannot sign blocks or votes. Validator keys are never placed in RPC client bundles. The public testnet plan uses a fresh genesis and a fresh key ceremony; internal test credentials are not reused.

5 Accounts and State

5.1 Accounts

An account is identified by a 20-byte address and stores:

- **balance**: spendable base units;
- **nonce**: the next accepted sequence number for fee-paying transactions;
- optional public-key metadata used by wallet and indexing surfaces.

The state also records:

- spent stealth key images;
- block and header metadata;
- the committee view for the current epoch;
- tokenomics counters: emitted supply, burned fees, distributed fees;
- snapshot identifiers and state roots.

5.2 Value and units

WIZM has eight decimal places. One WIZM is 10^8 base units. The current supply target is 8,312,029,999,000,000 base units, or 83,120,299.99 WIZM. The minimum transfer amount and the ingress balance floor are both 1,000,000 base units, or 0.01 WIZM.

5.3 State root

The executor maintains a canonical state root over account and protocol state. The root is persisted with each committed block. A node that imports a snapshot verifies the snapshot identifier and state root before accepting the state; a restart replays stored blocks from the executor resume point and checks the resulting root against the persisted chain.

6 Transactions

All integers are unsigned little-endian. Falcon public keys are 897 bytes and signatures are variable-length. Transaction classification is length-based with a binding check in the overlapping size band.

6.1 Transparent transfer

The transparent layout is

```
[from:20][to:20][amount:8][nonce:8]
[Falcon pk:897][signature:512..752]
```

Properties:

- wire size: 1,465–1,705 bytes;
- sender, recipient, and amount are public;
- the signature covers the 56-byte body `from || to || amount || nonce`;
- execution checks the account nonce, the sender balance, and the amount-plus-fee cost.

6.2 Stealth transfer

The stealth layout is

```
[sender:20][stealth_addr:20][ephemeral:32][view_tag:1]
[amount:8][key_image:32][Falcon pk:897][signature:512..752]
```

Properties:

- wire size: 1,522–1,762 bytes;
- the sender and amount are public; only the recipient link is hidden;
- `stealth_addr` is a one-time recipient address;
- the 113-byte body is signed;
- the key image prevents a second spend of the same stealth output;
- the amount is an unencrypted 8-byte consensus field.

For a recipient meta-address with spend public key pk_s, the sender samples an ephemeral secret r and publishes the ephemeral public value. The current derivation is

$$shared = \text{BLAKE3}(ephemeral), \tag{2}$$

$$stealth_addr = \text{BLAKE3}(pk_s \parallel shared)[0..20], \tag{3}$$

$$view_tag = shared[0]. \tag{4}$$

The recipient uses the view secret and the published meta-address to recognize candidate outputs. The one-time address is fresh for each transfer, so an observer cannot directly associate the payment with the published reusable address. The sender remains visible in the transaction, and the amount is visible in the clear. A key image is derived from the ephemeral value and the one-time address; the executor rejects a previously spent key image.

6.3 Transaction classification

The executor classifies transactions by size. Transparent and stealth size bands overlap because Falcon signatures are variable-length. In the overlap band the executor parses the embedded public key and checks whether addr(pk) equals the first 20 bytes. If it does, the transaction is treated as transparent; otherwise it is treated as stealth. Signature verification, address binding, and key-image checks remain the final gates.

6.4 Admission policy

Before a transaction enters the mempool, the RPC layer checks:

- the authentication and fee-payer permissions;
- the exact size envelope and minimum transfer amount;
- the fee payer's live balance against the ingress minimum;

- duplicate transaction hashes and previously seen raw bytes;
- the signature and address binding;
- the stealth key image, when applicable;
- per-IP and per-key rate limits.

7 Encrypted Mempool

7.1 Entry format

A mempool entry carries an encrypted transaction body and the metadata needed for ordering:

- a ciphertext produced with the committee ML-KEM-512 public key and AES-256-GCM;
- a sender/author identifier and a per-sender nonce where the layout exposes one;
- the arrival timestamp and admission metadata;
- the raw transaction hash used for deduplication.

The ciphertext protects transaction contents from observers until ordering. The plaintext account state, amounts, and stealth fields are public once a block is sealed; the encrypted mempool is not a substitute for amount or sender hiddenness.

7.2 Admission and capacity

The default pool is bounded by 500 MiB of payload, 200,000 entries, and a 5,400-second TTL. Admission is also subject to a process RSS ceiling: when the node approaches the configured memory ceiling, the pool sheds expired and oldest entries instead of allowing an OOM kill. The mempool uses per-sender nonce floors so that a missing nonce parks later entries instead of allowing an unbounded gap.

7.3 Gap buffering

If a submitted nonce is ahead of the sender's current floor, the entry is parked. The node remembers the proven contiguous prefix and releases parked entries when the gap is filled. A bounded hold timeout prevents an unresolvable gap from pinning memory forever. The exact policy is expressed as a transition system in the yellow paper.

7.4 Ordering and draining

A producer drains entries under the block count and byte caps. The drain order is deterministic for a given mempool state and is designed to preserve per-sender nonce order. The mempool exposes counters for admitted, deduplicated, rejected, evicted, parked, and forfeited entries so that silent loss is visible to operators.

8 Consensus

8.1 Committee

The committee is a fixed list of five seats with stable indices. Each seat has a Falcon-512 public key and a network address. A committee has an epoch number; epoch boundaries occur every 360 heights. The current deployment keeps the same five seats for the test period.

For a committee C of length N, the producer for height h at attempt a is

$$\text{producer}(h, a) = C[(h + a) \bmod N]. \tag{5}$$

Attempt 0 is the normal rotation. If a slot expires without a valid block, the same height is retried at the next attempt, using the next producer in the rotation. The fallback interval is 60 seconds.

8.2 Slots

The steady-state slot is 10 seconds. The genesis slot is 240 seconds and the next nine slots are 60 seconds. The slow start gives the peer-to-peer mesh time to form before competing producers can create same-height versions. A slot has a four-second grace window after its nominal end for late-but-valid blocks.

8.3 Block validity

A block is accepted only when:

- its height is the current tip plus one;
- its parent hash matches the current tip;
- its producer is the scheduled producer for some attempt that is due;
- the producer signature verifies against the committee key;
- the transaction count is at most 1,200;
- the body is at most 8 MiB;
- every transaction passes the execution and fee-payer checks;
- the transaction root matches the body.

8.4 Seal suppression gates

A restarted or lagging node must not seal while the canonical chain is still propagating. The node applies bounded suppression gates:

- post-boot mesh settle: no own proposals for 150 seconds after boot;
- first-seen buffering: no competing seal while a fresh same-height proposal is held, bounded by a 120-second window;
- behind-mesh gating: suppress local seals when a validated mesh sample is ahead, with a time-bounded contact rule;
- sync/catch-up gating: a node that is far behind executes and syncs instead of proposing;
- rebroadcast: a producer whose block was lost re-broadcasts the same bytes after a 60-second delay rather than minting a new version.

These gates are liveness aids, not extra voting rules. They prevent a half-formed mesh from producing permanent same-height forks.

8.5 Fork convergence

Two valid versions of the same height can arise across fallback boundaries or during network partitions. The convergence rule is:

1. the version with the lower attempt number wins;

2. if the attempt numbers are equal, the version with the lexicographically lower block hash wins.

The rule strictly improves the selected tip and is implemented with a bounded, deterministic unwind. Nodes verify the parent, producer signature, and transaction root before considering a competing version.

8.6 Finality

Validators sign notarization votes over

$$(\text{domain},\ \text{block hash},\ \text{height}).$$

For five active seats the quorum is

$$q = \left\lfloor \frac{5}{2} \right\rfloor + 1 = 3. \tag{6}$$

The notary records the first vote per signer and height. The finalized watermark advances only contiguously: height h is finalized when at least q valid votes exist for one hash at h and all lower heights are already finalized. A node that restarts may fast-forward its recovery watermark to $tip - 128$ because the stored prefix is validated; the protocol does not claim that this recovery floor is equivalent to a fresh quorum vote for every historical height.

The model is crash-fault finality. It provides deterministic progress under the stated validator assumptions, but it is not a Byzantine-fault-tolerant state-machine replication protocol.

9 Networking and Synchronization

9.1 Transport

Validators and full nodes use libp2p 0.57 over TCP and QUIC. The default P2P ports are 7777/TCP and 7778/UDP. Gossip topics cover transactions, blocks, votes, and synchronization control. Bulk block transfers use direct unicast where possible; gossip remains the secondary path.

9.2 Authenticated envelopes

Every consensus message is carried inside a post-quantum authenticated envelope. Envelope verification happens before decoding the inner message. Domain separation binds the chain ID and genesis hash, preventing a message from one network or fork being replayed on another.

9.3 Synchronization

A lagging node requests bounded block ranges from peers. The sync path:

- requests a contiguous prefix rather than a single far-future height;
- validates parent links, producer signatures, transaction roots, and execution before adoption;
- adopts a tip only from locally validated blocks;
- pipelines multiple ranges with a bounded lead;
- uses snapshots for large gaps when a quorum-consistent snapshot is available.

A node that has no useful local history can restore a snapshot, restart the executor from the snapshot height, and replay stored blocks to the tip. The snapshot and state root are checked before the state is trusted.

9.4 Peer limits

Each node caps its connected peer set at 128. The gossipsub mesh target is four peers with a high-water mark of six for each topic. These limits keep memory and fan-out bounded; they are not a global network limit. A public deployment should monitor peer counts and keep normal operation well below the hard cap.

10 Storage

RocksDB stores block bodies, headers, account state, stealth key images, snapshots, and metadata in separate column families. The current retention policy keeps:

- block bodies for 4,000 heights;
- headers for 200,000 heights;
- snapshots at a 2,000-height interval;
- a 32-block rewind journal for deterministic fork recovery.

State changes are written in atomic batches. The node persists the cumulative burned-fee counter, state root, account balances and nonces, stealth entries, and block metadata at commit time. Graceful shutdown syncs the WAL before the process exits. On restart the executor replays from its resume point and reconstructs the in-memory state before new seals are accepted.

Snapshots are used for fast onboarding and recovery. A snapshot export records the height, state root, and an identifier. An importer verifies the declared part count, slices, expected identifier, and state root before sealing the snapshot. A converged node refuses to import a snapshot, preventing an operator mistake from replacing live state with an older checkpoint.

11 RPC and API Surface

11.1 Transports

JSON-RPC listens on loopback by default and is normally exposed through a TLS reverse proxy. gRPC uses TLS when certificate and key paths are configured. Plaintext RPC is a local-only operator mode; it is not an internet-facing configuration.

11.2 Authentication

API keys carry four permissions:

- `can_submit_tx`: submit value-bearing transactions;
- `can_query`: read chain data;
- `can_admin`: administrative actions such as operator rewind;
- a per-key rate limit.

The current deployment separates load-test, wallet, and read-only keys. Keys are not embedded in client binaries. The operator rewind path requires an explicit admin permission and is not reachable with a normal submit key.

11.3 Rate limits and status behavior

Read calls consume a per-IP budget and a per-key budget when a key is present. The public edge budget is 500 requests per second per IP. Batch requests are capped at 100 items. The RPC layer maps authentication failures to HTTP 401, rate-limit failures to 429, oversized bodies to 413, and malformed UTF-8 to 400; JSON-RPC parse errors remain JSON-RPC errors. A 5 MiB body cap applies to the JSON-RPC listener; gRPC enforces its own message-size limit.

11.4 Endpoint families

The API exposes tip and block data, account balance and nonce, state root, peer count, mempool counters, tokenomics, snapshot metadata and slices, transaction submission, and stealth meta-address publication/scanning. The complete public method list is given in Appendix C.

12 Wallet Architecture

12.1 Native wallet

The native wallet stores key material under `~/.wizm/wallets` with directory mode `0700` and file mode `0600`. The Rust backend exposes explicit list, save, and delete commands. The wallet signs a canonical transaction body locally and submits only the signed bytes. Private keys do not enter the webview.

12.2 Browser wallet and sidecar

A browser wallet uses a loopback signing sidecar. The sidecar binds `127.0.0.1:18124` by default, requires a bearer token of at least 32 bytes, validates the HTTP Host and Origin, and issues an HttpOnly SameSite=Strict session cookie when serving the wallet page. The page's RPC calls use a short capability carried by the cookie; the node API key remains server-side. The sidecar allows node URLs only for the configured validator hosts and the loopback HTTP fallback, and it disables redirects for RPC requests. This prevents a page script from redirecting a request with the sidecar credential to an arbitrary host.

12.3 Signing boundary

The wallet's signing command accepts a secret key, public key, sender, recipient, amount, and nonce, checks that the public key derives the sender address, signs the canonical body, and returns the wire transaction. The submission path is separate. This split allows a wallet to sign offline and submit later, and it keeps the node's API key out of the signing command.

13 Tokenomics

13.1 Supply and emission

WIZM has 8 decimals and a total supply of 8,312,029,999,000,000 base units. The emission schedule spans ten years in five two-year periods:

Period	Years	Emission (WIZM)
1	1–2	22,248,474.30
2	3–4	20,023,626.87
3	5–6	17,020,082.84
4	7–8	13,616,066.27
5	9–10	10,212,049.71

The block reward per validator is

$$R = \frac{E_{\text{period}}}{B_{\text{period}}\, N_{\text{validators}}}, \tag{7}$$

where $B_{\text{period}} = 6{,}311{,}520$ blocks and $N_{\text{validators}} = 5$ in the current configuration. At the current period this is 88,126,450 base units, or 0.8812645 WIZM per validator per block.

13.2 Fees

The base fee is 10,000 base units (0.0001 WIZM). During emission the fee is burned. After the emission window the fee is divided among validators, with the indivisible remainder burned.

The fee for a wallet's n-th fee-paying transaction in one block is

$$f(n) = 10{,}000 \cdot 10^{\max(0,n-20)}, \tag{8}$$

with arithmetic saturation at the maximum representable fee. The first 20 transactions from a wallet in a block pay the base fee; the 21st pays 10x, the 22nd 100x, the 23rd 1,000x, and so on. The rule bounds a single wallet's ability to monopolize a block and makes bulk submission behavior explicit.

13.3 Economic invariants

For each block, the executor records the exact fees charged for transactions that actually applied. It adds those fees to the burned or distributed counters according to the emission phase. Transactions that fail execution do not add fees. The supply is the emitted supply minus burned fees plus any undistributed accounting; the state transition function in the yellow paper states the exact counters.

14 Performance and Capacity

14.1 Theoretical limits

A block is limited by both count and bytes:

$$T \le 1{,}200, \qquad S \le 8 \cdot 1024^2. \tag{9}$$

With a 10-second steady-state slot,

$$\Theta_{\max} = \frac{1{,}200}{10} = 120 \text{ TPS}. \tag{10}$$

The count cap normally binds first for transparent and stealth transfers. The 8 MiB cap protects propagation and storage from oversized bodies.

The per-wallet burst-free window is 20 transactions per block. At least 60 distinct wallets are needed to fill a 1,200-transaction block without multiplied fees.

14.2 Test-network measurements

Engineering runs on the five-validator network have exercised sustained mixed transparent/stealth traffic with offered phases from 50 to 150 TPS and a sealed ceiling of 120 TPS. Representative runs report 60–95 TPS sustained with peaks near 117 TPS, full 1,200-transaction blocks, and end-to-end block execution in the low seconds. High offered phases intentionally build a mempool backlog above the 120 TPS seal rate; low phases drain it. The ten-day soak harness records submitted, chain-included, rejected, and timed-out counts separately so that a network timeout is not misreported as a chain rejection.

14.3 Scaling the node set

The consensus set is five validators. Additional full nodes and RPC gateways do not increase the sealed TPS; they serve reads, sync, and redundancy. Each node has a 128-peer hard cap. A 50-node public testnet therefore fits comfortably below the per-node peer limit when validators and seeds are used as bootstrap points. The recommended topology is five validators, five seeds, ten public RPC/full nodes, and thirty community observers.

14.4 Fault behavior

With five seats and quorum three:

- one offline validator: four active seats continue;
- two offline validators: three active seats continue;
- three offline validators: only two seats remain, so finality cannot advance until a third seat returns;
- a restarted validator syncs missing blocks and backfills its finality recovery floor.

These behaviors have been exercised by controlled isolation and restart tests. They are crash-fault behaviors, not a claim of Byzantine resilience.

15 Security Analysis

15.1 Threat model

We consider:

- a network adversary that can delay, drop, duplicate, or reorder messages;
- crash faults and restarts of validators;
- malicious RPC clients that attempt authentication bypass, oversized requests, replay, malformed parameters, or resource exhaustion;
- compromised wallet endpoints that attempt to extract node credentials or redirect sidecar traffic;
- operator error during snapshots, key rotation, and upgrades.

We do not assume that a majority of validators is Byzantine. The current protocol is permissioned and crash-fault oriented. A production deployment requires independent review of the implementation and the operational controls.

15.2 Authentication and domain separation

Every write path verifies a bearer key with the required permission. Submit, query, and admin permissions are distinct. Consensus messages are signed with Falcon-512 and bound to the chain domain, genesis hash, height or round, and block hash. A signature produced for one chain or fork does not verify on another. Replays of sealed transactions are rejected by hash, nonce, and key-image checks.

15.3 Resource attacks

The principal resource controls are:

- block count and byte caps;
- mempool byte, entry, TTL, and RSS limits;
- per-IP and per-key RPC limits;
- a 100-item batch cap and 5 MiB HTTP body cap;
- a 128-peer connection cap;
- per-wallet burst fees.

A public edge should additionally use TLS, a reverse proxy, network ACLs, and standard DDoS protection. Rate limits are a first line of defense, not a substitute for capacity planning.

15.4 Unlinkability boundary

Stealth transfers are unlinkable at the recipient-address layer: each payment uses a fresh one-time address and a key image prevents a second spend. The sender and amount remain public. Network-level metadata, timing, amount correlation, and wallet behavior can still reveal relationships. WIZM does not claim graph-level anonymity.

15.5 Residual risks

Residual risks include implementation defects in cryptographic libraries, validator key compromise, operator misconfiguration, public-RPC abuse, long-range chain history assumptions, and the absence of a formal machine-checked model. The paper's claims should be read together with the test suite, the audit record, and the deployment runbooks.

16 Testing and Operations

The Rust workspace contains unit and integration tests for cryptography, transaction layouts, admission and execution, tokenomics, mempool behavior, consensus state, storage, snapshots, and RPC authorization. The test suite covers both transparent and stealth layouts and exercises rejection paths as well as successful paths.

Operational testing includes:

- validator isolation and rejoin;
- one-, two-, and three-validator outages;
- restart and snapshot recovery;
- ingress authentication and rate-limit probes;
- malformed JSON, oversized bodies, invalid UTF-8, and batch limits;
- sustained mixed-transaction soak testing.

The test harness separates four counters: submitted, chain-included, rejected, and timed out. This matters because a client timeout is not a chain verdict. The ten-day soak uses deterministic phases, fixed funding wallets, and a per-wallet cap below the burst threshold. Refill and funding paths are part of the test surface: a refill wallet can itself violate the per-wallet burst rule if it is not capped, so the harness distributes refills and caps them per round.

For a public test network the recommended controls are a fresh genesis, a separate validator key ceremony, private validator RPC, public read/write gateways behind TLS, monitoring for finality and peer counts, snapshot publication, and a documented incident process for validator loss or chain halt.

17 Related Work

WIZM combines established ideas in a deliberately narrow payment design. Proof-of-authority systems use a known validator set and a deterministic producer schedule to obtain low-latency ordering without permissionless mining. Classical BFT protocols provide stronger Byzantine guarantees but require additional voting rounds and communication; WIZM instead states a crash-fault model and uses a five-seat majority. Post-quantum signature schemes such as Falcon provide the authentication primitive. KEM-based transport protects validator links. One-time address constructions provide recipient unlinkability without changing the public amount.

The design differs from general smart-contract chains in its fixed execution model: two transfer layouts, sequential nonces, deterministic fees, and a bounded mempool. That narrowness is intentional: it makes the capacity, failure modes, and operational surface inspectable.

18 Limitations and Future Work

The current system is permissioned and test-network oriented. Important open work includes:

- independent security review of the cryptographic and networking implementation;
- a supported observer mode for public full nodes;
- committee rotation and governance procedures for validator changes;
- light-client proofs and a formal state-machine model;
- broader adversarial testing of the mempool and sync paths;
- reproducible release and key-ceremony procedures.

The protocol does not hide sender or amount in stealth payments. It does not provide general computation. It does not claim BFT safety under arbitrary validator behavior. These boundaries are part of the design, not omissions to be papered over.

19 Conclusion

WIZM is a post-quantum payment chain with a small, explicit protocol: five proof-of-authority validators, ten-second slots, full blocks, two transfer families, a bounded encrypted mempool, deterministic fees, and a restart-safe storage and sync path. Its most distinctive property is not a single primitive but the integration: Falcon-512 authentication, ML-KEM key establishment, BLAKE3-derived addresses, a deterministic PoA schedule, and a one-time recipient address scheme are combined into a running system with measurable limits. Transparent transfers expose the graph; stealth transfers hide the recipient link while keeping sender and amount public. The ten-day soak and the public-testnet plan are the next steps toward an independently reviewed deployment.

A Protocol Parameters

Parameter	Value

Chain ID	7777
Validators	5 seats
Steady slot	10 s
Genesis slot	240 s
Slow-start slots	60 s, heights 2–10
Slot grace	4 s
Fallback interval	60 s
Epoch length	360 heights
Max transactions/block	1,200
Max block body	8 MiB
Sealed throughput ceiling	120 TPS
Finality quorum	3 of 5 active seats
Finality recovery floor	tip − 128
Burst-free txs/wallet/block	20
Base fee	10,000 base units
Ingress minimum	1,000,000 base units
Minimum transfer	1,000,000 base units
Mempool bytes	500 MiB
Mempool entries	200,000
Mempool TTL	5,400 s
Peer cap	128 connections
Gossip mesh target	4 (high 6)
JSON-RPC body cap	5 MiB
gRPC message cap	4 MiB
Public read budget	500 req/s per IP
Batch cap	100 items
Snapshot interval	2,000 heights
Body retention	4,000 heights
Header retention	200,000 heights
Rewind journal	32 blocks

B Transaction Layouts

B.1 Transparent

```
offset  size   field
0       20     from address
20      20     recipient address
40      8      amount (u64 LE)
48      8      nonce (u64 LE)
56      897    Falcon-512 public key
953     512..752  Falcon-512 signature
```

B.2 Stealth

```
offset  size   field
0       20     sender address
20      20     one-time stealth address
40      32     ephemeral public value
72      1      view tag
```

```
73      8      amount (u64 LE)
81      32     key image
113     897    Falcon-512 public key
1010    512..752  Falcon-512 signature
```

C RPC Methods

The public JSON-RPC surface includes:

- chain reads: `wizm_getTip`, `wizm_getBlock`, `wizm_getSealedHeader`, `wizm_getStateRoot`, `wizm_getChainId` `wizm_getTokenomics`;
- account reads: `wizm_getBalance`, `wizm_getNonce`;
- mempool and network: `wizm_getMempoolStats`, `wizm_getPeerCount`;
- submission: `wizm_sendRawTransaction`, `wizm_sendCommitteeTx`, `wizm_sendSlashEvidence`;
- snapshot operations: `wizm_getSnapshotInfo`, `wizm_getSnapshotSlice`, `wizm_putSnapshotSlice`, `wizm_sealSnapshot`;
- stealth registry: `wizm_publishMetaAddress`, `wizm_scanStealth`;
- operator-only: `wizm_rewindTo`.

Write methods require the appropriate API-key permission. Public reads are rate-limited and may be served without a key according to the operator's edge configuration.

D Glossary

Attempt The retry index for a height after an empty slot.

Committee The ordered validator seats that sign blocks and votes.

Finalized The highest contiguous height with a quorum of matching notarization votes.

Key image A one-time value that prevents a second spend of a stealth output.

Meta-address The published spend/view key material used to derive one-time stealth addresses.

Slot A wall-clock production window, normally ten seconds.

Stealth address A one-time recipient address derived for a single transfer.

References

[1] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Specification, 2020.

[2] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé. *CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM*. IEEE EuroS&P, 2018.

[3] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. 2020.

[4] Protocol Labs. *libp2p: A Modular Peer-to-Peer Networking Stack*. 2024.

[5] Facebook Engineering. *RocksDB: A Persistent Key-Value Store for Flash and RAM Storage*. 2024.

[6] S. De Angelis, L. Aniello, R. Baldoni, F. Lombardi, A. Margheri, and V. Sassone. *PBFT vs Proof-of-Authority: Applying the CAP Theorem to Permissioned Blockchain*. 2018.

[7] N. van Saberhagen. *CryptoNote v2.0*. 2013.